ScriptDoctor: Automatic Generation of PuzzleScript Games via Large Language Models and Tree Search

Sam Earle*, Ahmed Khalifa†, Muhammad Umair Nasir‡, Zehua Jiang*, Graham Todd*,
Andrzej Banburski-Fahey§, Julian Togelius*
*New York University, Brooklyn, USA
{sam.earle, zj2086, gdt9380}@nyu.edu, julian@togelius.com
†University of Malta, Msida, Malta
ahmed@akhalifa.com
‡University of the Witwatersrand, Johannesburg, South Africa.
muhammad.nasir@witz.ac.za
§Microsoft, Redmond, USA.
abanburski@microsoft.com

***Abstract*—There is much interest in using large pre-trained models in Automatic Game Design (AGD), whether via the generation of code, assets, or more abstract conceptualization of design ideas. But so far, this interest largely stems from the ad hoc use of such generative models under persistent human supervision. Much work remains to show how these tools can be integrated into longer-time-horizon AGD pipelines, in which systems interface with game engines to test generated content autonomously. To this end, we introduce ScriptDoctor, a Large Language Model (LLM)-driven system for automatically generating and testing games in PuzzleScript, an expressive but highly constrained description language for turn-based puzzle games over 2D gridworlds. ScriptDoctor generates and tests game design ideas in an iterative loop, where human-authored examples are used to ground the system's output, compilation errors from the PuzzleScript engine are used to elicit functional code, and search-based agents play-test generated games. ScriptDoctor serves as a concrete example of the potential of automated, open-ended LLM-based workflows in generating novel game content.**

***Index Terms*—Automatic Game Design, PuzzleScript, Large Language Models, Tree Search**

I. Introduction

Can large language models (LLMs) create complete games? Spend some time scrolling through the right social media feeds, and you will see ample examples of this happening. So, apparently, yes, they can. Yet, many questions are hard to answer. How good are these games? We do not yet have the capabilities to play and test games in general, so this requires human evaluation. With the very large number of small games in the training set of a modern LLM, there remains the suspicion that the generated games are at best a minor variation on something it has already seen. The lack of automated evaluation is also limiting our ability to understand *how* to make an LLM produce the games we want.

In this paper, we seek to shed light on some of these questions not by investigating unconstrained game generation in JavaScript or Python, but by using a model organism of sorts. We investigate LLM-based game generation in a domain-specific language that allows for the creation of complete games in a manageable amount of text, which allows for generating diverse high-quality games, which allows for automatic game-testing through AI-based game-playing. Our model organism is *PuzzleScript* [1], a domain-specific language for puzzle games. This has multiple advantages. PuzzleScript was created as a way of prototyping puzzle games, and has since been used by hundreds of game designers to create thousands of games, some of them quite good and not all of them puzzle games. Importantly, the PuzzleScript engine has a standard format for input and output, and is lightweight enough to allow rapid rollouts.

II. Related work

Generating game rules or complete games is relatively uncommon in PCG work [1]. This is due to the complexity of the task and the difficulty of algorithmically evaluating overall game quality. Most game generation is based on the search-based paradigm [2] using playability information yielded from simulation and/or expert knowledge. A number of papers describe attempts to generate game rules or full games with various degrees of success [3]–[9].

Particularly relevant to this paper is [10], which generates PuzzleScript games. They fixed the number of game objects, and used a genetic algorithm to evolve a small subset of game rules. They use a constructive level generator that takes the generated rules and uses expert knowledge to place objects. The final levels and rules are evaluated using an automated player and compared to expert knowledge about the domain and game player data. The results were limited, and the possibility space of generated games depends on the hand-crafted constructive algorithm. The authors argue that having a better orchestration [11] between levels and rules using co-evolution [12] might improve the results.

With the rise of LLMs [13], one might think that the models would have some heuristics, perhaps implicit, for

Short Paper

[1]https://www.puzzlescript.net/

2025 IEEE Conference on Games (CoG) | 979-8-3315-8904-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/COG64752.2025.11114269

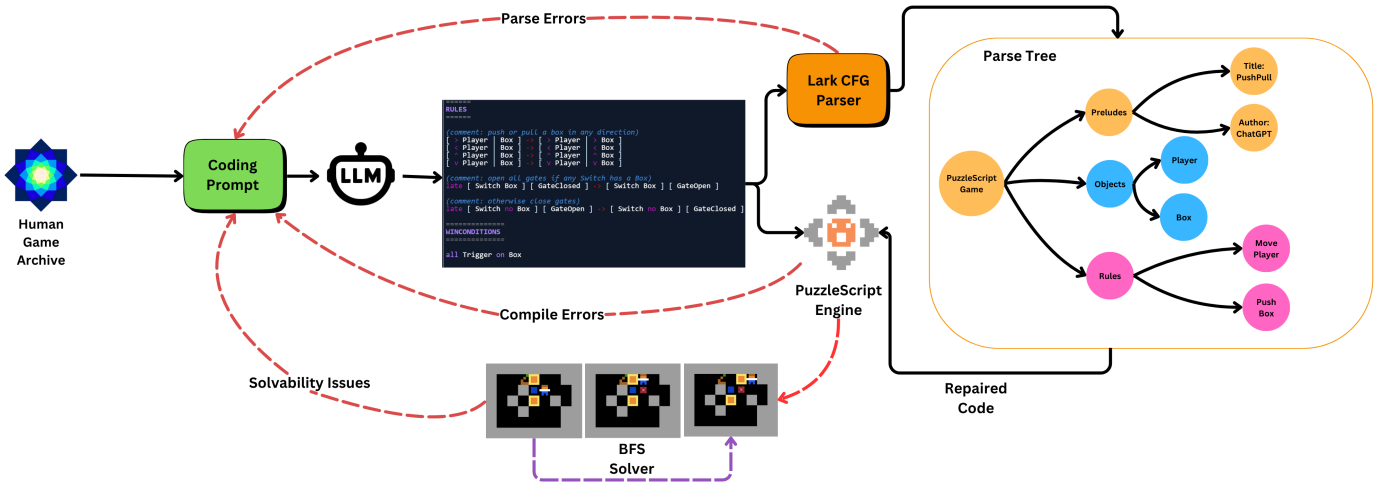

Fig. 1: **The ScriptDoctor automatic game generation pipeline.** An LLM iterates on PuzzleScript code given feedback from the compiler and a search-based player agent. Its output is parsed and repaired where possible according to a context-free grammar, and its prompt is augmented by examples of human games.

what makes a game good. This pushed researchers to explore the capabilities of these models in games [14]. [15] few-shot prompt LLMs to generate a simple maze game for the General Video Game framework [16]. [17] leverage LLMs' understanding of programming languages to generate arcade games written in a simple JavaScript game engine. [18] mix search-based methods [2] with LLMs to generate new games in the Ludii framework using a fine-tuned model.

This work sets itself apart from previous ones as it is not just producing game rules but also levels, and to some extent, visual aesthetics and narrative. We attempt to give LLMs the power to orchestrate between several different facets of the game [11] and produce a full PuzzleScript game in an unconstrained fashion. While this work investigates how the pre-trained knowledge of LLMs can be leveraged out of the box to support designers/developers.

III. Methods

Our system is developed as a fork of the original PuzzleScript repository [2]. We communicate with the existing game engine via a Python server. The server is responsible for controlling the hyperparameters of the game generation process and querying LLMs. On the client side, we compile the generated PuzzleScript games, and compilation errors or warnings from the engine are returned as potential feedback for the LLM. If the game compiles, each level is tested for playability using a breadth-first search algorithm. The algorithm runs until a solution is found or it explores 1M unique nodes. The results (solvability, number of expanded nodes, and solution length) from the BFS algorithm are sent back to the server to help with the generation process. A schematic overview of the system is depicted in Figure 1.

[2]https://github.com/increpare/PuzzleScript

The language model is given 10 total chances to output functional code. A generated script is deemed successful when it compiles successfully and each of its levels admits a solution of length > 10. At this point, the game generation trial is terminated. Otherwise, the LLM is prompted with the code that it generated at the prior iteration, any compilation warnings and errors that appeared in the PuzzleScript console, and feedback from the solver. We also define PuzzleScript as a Context-Free Grammar (CFG) using the Lark library for Python [3], and additionally include any syntax errors identified by this processing as feedback to the puzzle-generating LLM.

IV. Experiments

We experimented with generating games with zero-shot and few-shot prompting. In few-shot prompting, we sampled a group of random human-made games from a small dataset (610) of PuzzleScript games. The sampling continues until we reach the LLM context size, and adding another game will surpass the maximum number of tokens. This dataset is scraped from an online archive[4]. We also experimented with chain-of-thought reasoning to see the effect of this technique on the generated games. We tested different LLM models (GPT-4o [19], o1 [20], and o3-mini [21]) with varying context length ($10,000$, $30,000$, $50,000$, and $70,000$).

To asses the generated games, we measure the percentage of games that compile successfully in the PuzzleScript editor. We also measure the percentage of generated games wherein **any** level admits a solution of > 1 moves, and the percentage of generated games wherein **all** levels admit solutions of > 10 moves. (We note for context that many human-authored games admit short solutions while still being interesting or challenging.) To assess the **complexity** of generated solutions,

[3]https://github.com/lark-parser/lark
[4]https://pedros.works/puzzlescript/database

TABLE I: Effect of few-shot prompting GPT-4o by including a random subset of high-quality human-authored games in the prompt during code generation and repair (up to 30k tokens worth). Results aggregated over 20 game generation trials.

Fewshot	CoT	Compiles	Any Solvable	All Solvable	Sol. Complexity
F	F	30%	0%	0%	0 ± 0
	T	25%	10%	**5%**	13 ± 45
T	F	70%	40%	0%	**1,709** ± 6,722
	T	**80%**	**55%**	**5%**	1,498 ± 6,294

TABLE II: Comparison between models, each generating 10 games using fewshot prompting and a context of 10k tokens.

Model	Compiles	Any Solvable	All Solvable	Sol. Complexity
GPT-4o	67%	40%	**7%**	19 ± 27
o1	87%	**67%**	**7%**	**22,771** ± 84,485
o3-mini	**93%**	47%	20%	2,676 ± 9,888

we consider the number of nodes explored by the BFS player agent.

V. Results

In Table I, we observe the effect of few-shot prompting GPT-4o by including human-authored games in the model's system prompt and chain-of-thought prompting it. We see that few-shot prompting increases the quality of generated code in terms of the proportion of generated games that successfully compile and the percentage of games that are solvable via tree search. It likewise results in increased complexity of solutions to generated games.

In Table II, we compare different LLM models that are used in ScriptDoctor and find that reasoning models o1 and o3-mini outperform GPT-4o in terms of the functionality and complexity of generated games. (This aligns with the improvement elicited by chain-of-thought prompting in Table I.) A particularly challenging mechanic and level design by o1 is featured in Figure 2

We also compare the effect of different context lengths on GPT-4o in Table III. We notice that increasing context length allows the system to generate more compilable games, which might be due to the effect of having more games in the prompt. We also notice that the quality doesn't change that drastically after $30,000$ tokens. We believe that after a certain point of context length, having more games doesn't add much to the model's understanding of what it needs to create. Future work might be needed to investigate what can be provided to improve the generated games besides adding few-shot examples.

VI. Discussion

The strong positive effect of a few-shot prompting the system with human games is unsurprising. PuzzleScript is not nearly as prevalent as a coding language, such as Python or C++, so having human examples is crucial to facilitate the model's outputting syntactically correct and compilable code. Without finetuning, LLMs struggle with the kind of spatial reasoning at work in level generation [22], [23]. Often, ScriptDoctor outputs levels that admit overly short solutions. When asked to complexify their solutions, it attempts to edit the level layout to make it more challenging, but often this only amounts to stretching the level out by a few rows or columns, or adding some scattered obstacles.

TABLE III: Effect of increasing GPT-4o's context window when including human-authored games in the prompt.

Context Length	Compiles	Any Solvable	All Solvable	Sol. Complexity
10,000	60%	30%	**10%**	18 ± 28
30,000	**100%**	**80%**	0%	46 ± 44
50,000	90%	40%	**10%**	**7,003** ± 19,329
70,000	**100%**	40%	0%	203 ± 553

The output of our system may serve as a dataset for fine-tuning smaller, more accessible LLMs. Given that our pipeline verifies the compilability and solvability of generated games, it could provide a suitable dataset for supervised fine-tuning such models (alongside the dataset of human-authored games). Having a fine-tuned model would allow us to perform constrained generation of PuzzleScript code using our Context-Free Grammar specification of the PuzzleScript language, effectively guaranteeing the syntactical correctness of generated code similar to [18].

While information about solvability is valuable for an iterative code-generating LLM, it may well be insufficient for capturing whether mechanics are "broken" relative to the model's intent. In fact, we observe that the most complex games tend to be solvable or complex despite or even as a result of their broken mechanics (e.g. Figure 3). For the system to reliably fix such issues, some more targeted form of feedback is likely necessary. One option could be to collect frames of gameplay in which particular rules were applied, and supply a vision-language model with the appropriate context to be able to diagnose potential flaws in their implementation.

VII. Conclusion

Despite increasing interest in generating arbitrary video games with LLMs via quasi-passive "vibe coding", there exist few means for quantitatively evaluating these models' ability to produce novel and functional games autonomously. We answer the call with ScriptDoctor, an automatic game designer that integrates LLMs with search-based player agents and grammatical parse trees to generate concise but expressive games in PuzzleScript. The system operates in a possibility space that is both open-ended—with generated scripts defining everything from sprites, to mechanics, to levels—and tractable for artificial playtesting—with a small, fixed action space.

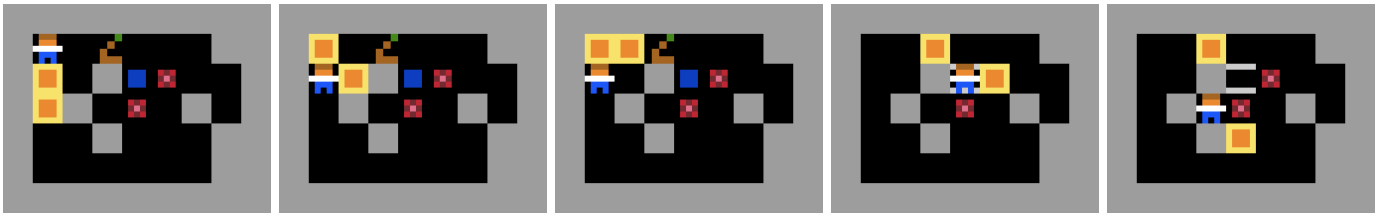

Fig. 2: Select frames from a solution to the *Unconventional PushPull* puzzle generated by ScriptDoctor. The player must place crates (gold boxes) on targets (red squares), by pushing and pulling them, as well as "sliding" them one tile beyond the player's immediate neighborhood when the player is obstructed in the relevant direction. The switch (brown lever) can be activated by crates in order to make the gate (blue) traversible. The most direct solution is 34 moves long. To a human player, the solution is tricky but sensible. For the sake of visualization, we replace generated sprites with sprites from human-authored games.

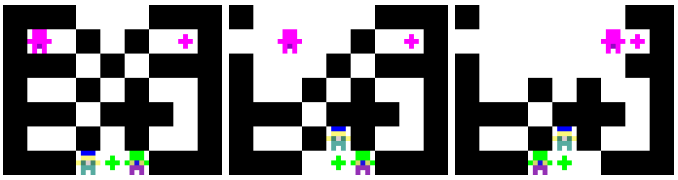

Fig. 3: Select frames from a level of a generated game by GPT-4o. This level involves simultaneous control of 2 characters. The wizard (pink) can teleport through walls if they aren't adjacent or destroy them otherwise without moving. Taking advantage of the remove mechanic, the solver can reposition the rogue (green) such that one final move to the right will result in both the wizard and the rogue reaching their targets.

We find that using few-shot prompting with examples of human-authored games increases the functionality of generated code. We find that frontier reasoning models o1 and o3-mini are more capable than their non-reasoning counterpart GPT-4o in generating compilable and playable games. Increasing the context length to include more human-authored games shows diminishing returns after $30,000$ tokens, suggesting that the model might need more information to understand how to reason beyond these examples.

Future work can leverage ScriptDoctor's automatic metrics of quality to guide the generative process, wrapping it in a novelty-seeking evolutionary loop. PuzzleScript's pattern rewrite rules lend themselves both to static causal analysis and reimplementation via GPU-compatible convolutions for accelerated playtesting—both potentially valuable signals during the open-ended search for novel video games.

References

[1] G. N. Yannakakis and J. Togelius, *Artificial intelligence and games*. Springer, 2018, vol. 2.

[2] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Search-based procedural content generation: A taxonomy and survey," *Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, 2011.

[3] C. B. Browne, "Automatic generation and evaluation of recombination games," Ph.D. dissertation, Queensland University of Technology, 2008.

[4] J. Togelius and J. Schmidhuber, "An experiment in automatic game design," in *Symposium On Computational Intelligence and Games*. IEEE, 2008.

[5] M. Treanor, B. Blackford, M. Mateas, and I. Bogost, "Game-o-matic: Generating videogames that represent ideas," in *Procedural Content Generation in Games Workshop*, 2012.

[6] M. Cook, S. Colton, A. Raad, and J. Gow, "Mechanic miner: Reflection-driven game mechanic discovery and level design," in *EvoApplications, EvoStar Conference*. Springer, 2013.

[7] T. S. Nielsen, G. A. Barros, J. Togelius, and M. J. Nelson, "Towards generating arcade game rules with vgdl," in *Computational Intelligence and Games Conference*. IEEE, 2015.

[8] A. Khalifa, M. C. Green, D. Perez-Liebana, and J. Togelius, "General video game rule generation," in *Computational Intelligence and Games Conference*. IEEE, 2017.

[9] J. J. Gonzalez, S. Cooper, and M. Guzdial, "Mechanic maker 2.0: reinforcement learning for evaluating generated rules," in *Artificial Intelligence and Interactive Digital Entertainment Conference*, vol. 19, no. 1, 2023.

[10] A. Khalifa and M. Fayek, "Automatic puzzle level generation: A general approach using a description language." IEEE, 2015.

[11] A. Liapis, G. N. Yannakakis, M. J. Nelson, M. Preuss, and R. Bidarra, "Orchestrating game generation," *Transactions on Games*, vol. 11, no. 1, 2018.

[12] X. Ma, X. Li, Q. Zhang, K. Tang, Z. Liang, W. Xie, and Z. Zhu, "A survey on cooperative co-evolutionary algorithms," *Transactions on Evolutionary Computation*, vol. 23, no. 3, 2018.

[13] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[14] R. Gallotta, G. Todd, M. Zammit, S. Earle, A. Liapis, J. Togelius, and G. N. Yannakakis, "Large language models and games: A survey and roadmap," *arXiv preprint arXiv:2402.18659*, 2024.

[15] C. Hu, Y. Zhao, and J. Liu, "Game generation via large language models," in *Conference on Games*. IEEE, 2024.

[16] D. Perez-Liebana, J. Liu, A. Khalifa, R. D. Gaina, J. Togelius, and S. M. Lucas, "General video game ai: A multitrack framework for evaluating agents, games, and content generation algorithms," *Transactions on Games*, vol. 11, no. 3, 2019.

[17] K. Cho, "Can ai chatbots create new games?" https://abagames.github.io/joys-of-small-game-development-en/generation/can_ai_chatbot_create_game.html, 2024, accessed on: October 23, 2024.

[18] G. Todd, A. Padula, M. Stephenson, É. Piette, D. J. Soemers, and J. Togelius, "Gavel: Generating games via evolution and language models," *arXiv preprint arXiv:2407.09388*, 2024.

[19] OpenAI, "Hello gpt-4o," May 2024, accessed: 2024-09-11. [Online]. Available: https://openai.com/index/hello-gpt-4o/

[20] A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Helyar, A. Madry, A. Beutel, A. Carney *et al.*, "Openai o1 system card," *arXiv preprint arXiv:2412.16720*, 2024.

[21] OpenAI, "Openai o3-mini system card," January 2025, accessed: March 15, 2025. [Online]. Available: https://openai.com/index/o3-mini-system-card/

[22] S. Sudhakaran, M. González-Duque, M. Freiberger, C. Glanois, E. Najarro, and S. Risi, "Mariogpt: Open-ended text2level generation through large language models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[23] G. Todd, S. Earle, M. U. Nasir, M. C. Green, and J. Togelius, "Level generation through large language models," in *Foundations of Digital Games Conference*, 2023.